EE120 Final Project Report

Networked RGB Lamp System (Arduino + ESP8266)

Gabriel Jordaan, Sam Schultz, Nick Sankovich

May 2026

1 Project Overview

This project implements a network of four RGB lamps that communicate via a mesh of ESP8266 Wi-Fi nodes paired with Arduino Uno boards. Each lamp has a sensor that, when activated, signals its adjacent lamps over the network to turn on for a fixed duration. Each lamp consists of two microcontrollers: an Arduino Uno (sensor / RGB driver / coordinator) and an ESP8266 (Wi-Fi transport, with one node configured as a router/access-point and the rest as clients).

2 Source Code

2.1 Arduino: `ardu code/src/main.cpp`

```cpp
#include <Arduino.h>

#include "Internet.h"
#include "SignalCoordinator.h"
#include "Light.h"
#include "Sensor.h"

//NUM_LIGHTS is defined in SignalCoordinator.h and in Internet.h so if you change one change the other... #HACK

#define LAMP_INDEX 2
#define RED 255
#define GREEN 255
#define BLUE 255

#define ARDU_TX_ADD_PIN 12
#define ARDU_TX_REMOVE_PIN 5

#define TIME 10

Internet internet;
SignalCoordinator signalCoordinator;
Light light;
Sensor sensor;

byte lamp_number = LAMP_INDEX;
byte color_r = RED;
byte color_g = GREEN;
byte color_b = BLUE;

```

```
byte seconds_active = 0;
unsigned long int last_update_time = 0;

void setup() {

  Serial.begin(9600);

  internet.begin();
  signalCoordinator.begin(internet);
  light.begin();
  sensor.begin();

  last_update_time = millis();
}

void loop() {

  internet.update();
  signalCoordinator.update();
  light.update();
  sensor.update();

  if (sensor.isActive()){
    seconds_active = TIME;
    signalCoordinator.txActiveSignal();
    last_update_time = millis();
    Serial.println("Sensor active");
  }
  else if (signalCoordinator.rxActiveSignal(lamp_number)){
    seconds_active = TIME;
    last_update_time = millis();
    Serial.println("Received active signal");
    internet.txDeleteMessage();
  }
  else if (seconds_active > 0 && millis() - last_update_time > 1000){
    seconds_active--;
    last_update_time = millis();
    Serial.print("Seconds active: ");
    Serial.println(seconds_active);
  }

  if (seconds_active > 0){
    light.setColor(color_r, color_g, color_b);
    Serial.println("Light on");
  }
  else{
    light.setColor(0, 0, 0);
  }

}

int main() {
  init();
  setup();
  while (true) {
    loop();
  }
  // Never happens
  return 0;
```

```
}
```

2.2 Arduino: lib/Internet/src/Internet.h

```
#pragma once
#include <Arduino.h>

#define NUM_LIGHTS 4 // HACK: This is defined in both Internet.h and SignalCoordinator.h so if you change one change the other.

#define ARDU_TX_ADD_PIN 12
#define ARDU_TX_REMOVE_PIN 5
# define ARDU_RX_PIN 6

class Internet {
 public:
  Internet();

  void begin();
  void update();
  void txMessage();
  bool rxMessage();
  void txDeleteMessage();

 private:
  static unsigned long started_message_at;
  static bool is_sending_message;
  static unsigned long started_delete_message_at;
  static bool is_sending_delete_message;
};
```

2.3 Arduino: lib/Internet/src/Internet.cpp

```
//Gabe
//txMessage appends the target_number to the end of the message queue removing duplicates.
//rxMessage copies the message queue into the provided array and sets size to the number of messages in the queue.

#include "Internet.h"

Internet::Internet() {}

unsigned long Internet::started_message_at = 0;
bool Internet::is_sending_message = false;

unsigned long Internet::started_delete_message_at = 0;
bool Internet::is_sending_delete_message = false;

void Internet::begin() {
    pinMode(ARDU_TX_ADD_PIN, OUTPUT);
    pinMode(ARDU_TX_REMOVE_PIN, OUTPUT);
    pinMode(ARDU_RX_PIN, INPUT);
}

```

```
void Internet::update() {
  if(is_sending_message and millis() - started_message_at > 1000){
    digitalWrite(ARDU_TX_ADD_PIN, LOW);
    is_sending_message = false;
  }
  if(is_sending_delete_message and millis() - started_delete_message_at > 1000){
    digitalWrite(ARDU_TX_REMOVE_PIN, LOW);
    is_sending_delete_message = false;
  }
}

void Internet::txMessage() {
    if (is_sending_message) return;
    is_sending_message = true;
    digitalWrite(ARDU_TX_ADD_PIN, HIGH);
    started_message_at = millis();
}

bool Internet::rxMessage() {
    return digitalRead(ARDU_RX_PIN) == HIGH;
}

void Internet::txDeleteMessage() {
    if (is_sending_delete_message) return;
    digitalWrite(ARDU_TX_REMOVE_PIN, HIGH);
    is_sending_delete_message = true;
    started_delete_message_at = millis();
}
```

2.4 Arduino: lib/SignalCoordinator/src/SignalCoordinator.h

```
#pragma once
#include "Internet.h"
#include "Utils.h"
#include <Arduino.h>

#define NUM_LIGHTS 4 // HACK: This is defined in both Internet.h and
    SignalCoordinator.h so if you change one change the other.



class SignalCoordinator {
  public:
  SignalCoordinator();

  void begin(Internet& internet);
  void update();
  bool rxActiveSignal(byte lamp_number);
  void txActiveSignal();

  private:
  Internet* internet_ptr;
  bool prev_in_list;
};
```

2.5 Arduino: lib/SignalCoordinator/src/SignalCoordinator.cpp

```cpp
// Sam Schultz.
// Send signals to the other lights based upon which light it is.
// Use internet.txMessage(target_number) and internet.rxMessage() to send and receive messages.

#include "SignalCoordinator.h"

SignalCoordinator::SignalCoordinator() : internet_ptr(nullptr), prev_in_list(false) {}

void SignalCoordinator::begin(Internet& internet) {
    internet_ptr = &internet;
}

void SignalCoordinator::update() {}

bool SignalCoordinator::rxActiveSignal(byte lamp_number) {
    //int len = 0;
    //byte arr[NUM_LIGHTS];
    //internet_ptr->rxMessage(arr, &len);
    //bool in_list = in_in(arr, len, lamp_number);

    //if (in_list && !prev_in_list) {
    //    internet_ptr->txDeleteMessage();
    //    Serial.println("Received active signal, sending delete message");
    //}
    //prev_in_list = in_list;

    //return in_list;

    return internet_ptr->rxMessage();
}

void SignalCoordinator::txActiveSignal() {
    internet_ptr->txMessage();
}
```

2.6 Arduino: lib/Light/src/Light.h

```cpp
#pragma once
#include <Arduino.h>

#define RED_PIN 10
#define GREEN_PIN 11
#define BLUE_PIN 3

class Light {
 public:
  Light();

  void begin();
  void update();
  void setColor(byte r, byte g, byte b);
};
```

2.7 Arduino: lib/Light/src/Light.cpp

```cpp
//Nick Sankovich
//Light. Set the color.

#include "Light.h"

Light::Light() {}

void Light::begin() {
  pinMode(RED_PIN, OUTPUT);  //Set RGB pins as outputs
  pinMode(GREEN_PIN, OUTPUT);
  pinMode(BLUE_PIN, OUTPUT);
}

void Light::update() {}

void Light::setColor(byte r, byte g, byte b) {
  analogWrite(RED_PIN, r); //Set color intensities
  analogWrite(GREEN_PIN, g);
  analogWrite(BLUE_PIN, b);

}
```

2.8 Arduino: lib/Sensor/src/Sensor.h

```cpp
#pragma once
#include <string.h>

#define SENSOR_MODE "ANALOG" // "DIGITAL" or "ANALOG"
#define SENSOR_THRESHOLD 512 // Only used in ANALOG mode, the sensor is active if the value read from the sensor is above this threshold.

class Sensor {
 public:
  Sensor();

  void begin();
  void update();
  bool isActive();
};
```

2.9 Arduino: lib/Sensor/src/Sensor.cpp

```cpp
//Nick Sankovich.
//Return true if the sensor is active, false otherwise. There must be two modes "ANALOG", and "DIGITAL". In ANALOG mode, the sensor is active if the value read from the sensor is above a certain threshold. In DIGITAL mode, the sensor is active if the value read from the sensor is HIGH. The mode and threshold should be configurable in the constructor of the Sensor class.

#include "Sensor.h"
#include <Arduino.h>

#define SENSOR_ACTIVE_HIGH 0
```

```
#define SENSOR_MODE "ANALOG"
#define SENSOR_THRESHOLD 512
#define SENSOR_PIN_ANALOG A0
#define SENSOR_PIN_DIGITAL 13
Sensor::Sensor() {}

void Sensor::begin() {
  if (strcmp(SENSOR_MODE, "DIGITAL") == 0) {
    pinMode(SENSOR_PIN_DIGITAL, INPUT_PULLUP);
  } else {
    pinMode(SENSOR_PIN_ANALOG, INPUT_PULLUP);
  }
}

void Sensor::update() {}

bool Sensor::isActive() {
  if (strcmp(SENSOR_MODE, "DIGITAL") == 0) {
  int value = digitalRead(SENSOR_PIN_DIGITAL);

    if (SENSOR_ACTIVE_HIGH) {
      return value == HIGH;
    } else{
      return value == LOW;
    }
  }
  if (strcmp(SENSOR_MODE, "ANALOG") == 0){
    int value = analogRead(SENSOR_PIN_ANALOG);

    Serial.print("Analog value: ");
    Serial.println(value);

    if (SENSOR_ACTIVE_HIGH) {
      return value >= SENSOR_THRESHOLD;
    }  else{
       return value <= SENSOR_THRESHOLD;
    }
  }
  return false;
}
```

2.10 Arduino: lib/Utils/src/Utils.h

```
#pragma once
#include <Arduino.h>

bool in_in(byte array[], int size, int value);
```

2.11 Arduino: lib/Utils/src/Utils.cpp

```
#include "Utils.h"

bool in_in(byte array[], int size, int value) {
  for (int i = 0; i < size; i++) {
    if (array[i] == value) {
```

```
      return true;
    }
  }
  return false;
}
```

2.12 Arduino: platformio.ini

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:uno]
platform = atmelavr
board = uno
framework = arduino
```

2.13 ESP8266: ESP_code_EE120_final/src/main.cpp

```
#include <Arduino.h>
#include <ESP8266WiFi.h>

#define NUM_LIGHTS 4

#define IS_ROUTER 0

#define LIGHT_INDEX 2

// Adjacent light indices, comma-separated. Must contain at least one entry.
#define ADJACENT {1, 3}

#define ARDU_TX_ADD_PIN 12

#define ARDU_TX_REMOVE_PIN 5

#define ESP_RX_ADD_PIN 4

#define ESP_RX_REMOVE_PIN 16

#define ESP_TX_PIN 14

#define NET_SSID "EE120_NET"
#define NET_PASS "ee120pass"
#define NET_PORT 4210
#define MAX_CLIENTS 8

static const byte adjacent[] = ADJACENT;
static const byte adjacent_count = sizeof(adjacent) / sizeof(adjacent[0]);

```

```
static void updateSelfPin(const byte* list, byte count) {
  bool present = false;
  for (byte i = 0; i < count; i++) {
    if (list[i] == LIGHT_INDEX) { present = true; break; }
  }
  digitalWrite(ESP_TX_PIN, present ? HIGH : LOW);
}

#if IS_ROUTER

static byte numbers[NUM_LIGHTS];
static byte numbers_count = 0;

static WiFiServer server(NET_PORT);
static WiFiClient clients[MAX_CLIENTS];

static void addNumber(byte n) {
  for (byte i = 0; i < numbers_count; i++) {
    if (numbers[i] == n) return;
  }
  if (numbers_count < NUM_LIGHTS) {
    numbers[numbers_count++] = n;
  }
}

static void removeNumber(byte n) {
  for (byte i = 0; i < numbers_count; i++) {
    if (numbers[i] == n) {
      for (byte j = i; j + 1 < numbers_count; j++) numbers[j] = numbers[j + 1];
      numbers_count--;
      return;
    }
  }
}

static void broadcastList() {
  updateSelfPin(numbers, numbers_count);
  String msg = "LIST";
  for (byte i = 0; i < numbers_count; i++) {
    msg += ' ';
    msg += numbers[i];
  }
  msg += '\n';
  for (byte i = 0; i < MAX_CLIENTS; i++) {
    if (clients[i] && clients[i].connected()) {
      clients[i].print(msg);
    }
  }
}

static void handleCommand(const String& cmd) {
  if (cmd.startsWith("ADD ")) {
    addNumber((byte)cmd.substring(4).toInt());
    broadcastList();
  } else if (cmd.startsWith("REMOVE ")) {
    removeNumber((byte)cmd.substring(7).toInt());
    broadcastList();
  } else if (cmd == "LIST") {
    broadcastList();
```

```
  }
}

static void networkBegin() {
  WiFi.mode(WIFI_AP);
  WiFi.softAP(NET_SSID, NET_PASS);
  server.begin();
  server.setNoDelay(true);
}

static void networkUpdate() {
  if (server.hasClient()) {
    bool placed = false;
    for (byte i = 0; i < MAX_CLIENTS; i++) {
      if (!clients[i] || !clients[i].connected()) {
        if (clients[i]) clients[i].stop();
        clients[i] = server.accept();
        placed = true;
        break;
      }
    }
    if (!placed) {
      WiFiClient reject = server.accept();
      reject.stop();
    }
  }
  for (byte i = 0; i < MAX_CLIENTS; i++) {
    if (clients[i] && clients[i].connected() && clients[i].available()) {
      String line = clients[i].readStringUntil('\n');
      line.trim();
      if (line.length() > 0) handleCommand(line);
    }
  }
}

static void sendAdd(byte n) {
  addNumber(n);
  broadcastList();
}

static void sendRemove(byte n) {
  removeNumber(n);
  broadcastList();
}

#else  // !IS_ROUTER

static WiFiClient uplink;
static byte last_list[NUM_LIGHTS];
static byte last_list_count = 0;

static void networkBegin() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(NET_SSID, NET_PASS);
}

static void ensureUplink() {
  if (WiFi.status() != WL_CONNECTED) return;
  if (uplink.connected()) return;
```

```
  uplink.connect(WiFi.gatewayIP(), NET_PORT);
}

static void networkUpdate() {
  ensureUplink();
  while (uplink.connected() && uplink.available()) {
    String line = uplink.readStringUntil('\n');
    line.trim();
    if (!line.startsWith("LIST")) continue;
    byte parsed[NUM_LIGHTS];
    byte count = 0;
    String rest = line.substring(4);
    rest.trim();
    while (rest.length() > 0 && count < NUM_LIGHTS) {
      int sp = rest.indexOf(' ');
      String tok = (sp < 0) ? rest : rest.substring(0, sp);
      parsed[count++] = (byte)tok.toInt();
      if (sp < 0) break;
      rest = rest.substring(sp + 1);
      rest.trim();
    }
    last_list_count = count;
    for (byte i = 0; i < count; i++) last_list[i] = parsed[i];
    updateSelfPin(parsed, count);
  }
}

static void sendAdd(byte n) {
  ensureUplink();
  if (uplink.connected()) {
    uplink.print("ADD ");
    uplink.println(n);
  }
}

static void sendRemove(byte n) {
  ensureUplink();
  if (uplink.connected()) {
    uplink.print("REMOVE ");
    uplink.println(n);
  }
}

#endif

static bool prev_rx_add = false;
static unsigned long last_debug_ms = 0;

static void debugPrint() {
  unsigned long now = millis();
  if (now - last_debug_ms < 500) return;
  last_debug_ms = now;
  Serial.print("RX_ADD=");
  Serial.print(digitalRead(ESP_RX_ADD_PIN));
  Serial.print(" RX_REMOVE=");
  Serial.print(digitalRead(ESP_RX_REMOVE_PIN));
  Serial.print(" LIST=[");
#if IS_ROUTER
  for (byte i = 0; i < numbers_count; i++) {
```

```
    if (i) Serial.print(',');
    Serial.print(numbers[i]);
  }
#else
  for (byte i = 0; i < last_list_count; i++) {
    if (i) Serial.print(',');
    Serial.print(last_list[i]);
  }
#endif
  Serial.println("]");
}

void setup() {
  Serial.begin(115200);
  pinMode(ESP_RX_ADD_PIN, INPUT);
  pinMode(ESP_RX_REMOVE_PIN, INPUT);
  pinMode(ESP_TX_PIN, OUTPUT);
  digitalWrite(ESP_TX_PIN, LOW);
  networkBegin();
}

void loop() {
  networkUpdate();
  debugPrint();

  bool rx_add = digitalRead(ESP_RX_ADD_PIN) == HIGH;

  if (rx_add && !prev_rx_add) {
    for (byte i = 0; i < adjacent_count; i++) {
      sendAdd(adjacent[i]);
    }
  }
  if (digitalRead(ESP_RX_REMOVE_PIN) == HIGH) {
    sendRemove(LIGHT_INDEX);
  }

  prev_rx_add = rx_add;
}
```

2.14 ESP8266: `platformio.ini`

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:esp8266]
platform = espressif8266
board = nodemcu
framework = arduino
upload_speed = 115200
monitor_speed = 115200
```

2.15 Build / Macro Patcher: `macrocompile.py`

```python
#!/usr/bin/env python3
DEFAULT_MACROS = {

    "LAMP_INDEX": 2,
    "SENSOR_ACTIVE_HIGH": 0,
    "SENSOR_MODE": "\"ANALOG\"", # NOTE FOR AFTER LUNCH !!! FLASH NUMBER 2!!!

    "IS_ROUTER": 0,
    "ADJACENT": "",

    "NUM_LIGHTS": 4,

    "SENSOR_PIN_ANALOG": "A0", # For the sensor
    "SENSOR_PIN_DIGITAL": 13,

    "RED": 255,
    "GREEN": 255,
    "BLUE": 255,

    "RED_PIN": 10, # For outputting to the LED
    "GREEN_PIN": 11,
    "BLUE_PIN": 3,

    "ESP_RX_ADD_PIN": 4, # For the arduino to send a signal to the esp to add the adjacent lights to the network list (D2 = GPIO4)
    "ARDU_TX_ADD_PIN": 12,

    "ESP_RX_REMOVE_PIN": 16, # For the arduino to respond to the light timer signal with an ack to remove the light from the network list (D0 = GPIO16)
    "ARDU_TX_REMOVE_PIN": 5,

    "ESP_TX_PIN": 14, # For turning the light timer on from the ESP (D5 = GPIO14)
    "ARDU_RX_PIN": 6,

    "SENSOR_THRESHOLD": 512,

    "NET_SSID": "\"EE120_NET\"",
    "NET_PASS": "\"ee120pass\"",
    "NET_PORT": 4210,
    "MAX_CLIENTS": 8,

    "TIME": 10

}

match DEFAULT_MACROS["LAMP_INDEX"]:
    case 0:
        DEFAULT_MACROS["ADJACENT"] = "{1}"
        DEFAULT_MACROS["IS_ROUTER"] = 1
    case 1:
        DEFAULT_MACROS["ADJACENT"] = "{0, 2, 3}"
    case 2:
        DEFAULT_MACROS["ADJACENT"] = "{1, 3}"
    case 3:
        DEFAULT_MACROS["ADJACENT"] = "{1, 2}"

DEFAULT_MACROS["LIGHT_INDEX"] = DEFAULT_MACROS["LAMP_INDEX"]
```

```python

## This is a slop script to patch our hacky cross platform macro system.



import json
import re
import sys
from pathlib import Path

DEFAULT_ROOTS = ["ardu code", "ESP_code_EE120_final"]
SUBDIRS = ["src", "lib"]
EXTENSIONS = {".h", ".hpp", ".c", ".cpp", ".cc", ".ino"}

DEFINE_PATTERN = re.compile(
    r"^(\s*#\s*define\s+)(\w+)(\s+)(.*?)(\s*(?://|/\*).*)?$",
    re.MULTILINE,
)


class UnknownMacroError(Exception):
    pass


def patch_text(text: str, macros: dict, source: str = "<text>") -> tuple[str, list[tuple[str, str, str]]]:
    changes: list[tuple[str, str, str]] = []

    def _sub(m: re.Match) -> str:
        prefix, name, ws, old_value, comment = (
            m.group(1),
            m.group(2),
            m.group(3),
            m.group(4).rstrip(),
            m.group(5) or "",
        )
        if name not in macros:
            raise UnknownMacroError(
                f"{source}: #define {name!r} is not listed in DEFAULT_MACROS "
                f"(add it with a value, or set it to None to leave it unmodified)"
            )
        new_value = macros[name]
        if new_value is None:
            return m.group(0)
        new_value_str = str(new_value)
        if old_value == new_value_str:
            return m.group(0)
        changes.append((name, old_value, new_value_str))
        return f"{prefix}{name}{ws}{new_value_str}{comment}"

    text = DEFINE_PATTERN.sub(_sub, text)
    return text, changes


def iter_files(roots: list[Path]):
    for root in roots:
        if not root.exists():
            continue
        for sub in SUBDIRS:
```

```
            base = root / sub
            if not base.exists():
                continue
            for path in base.rglob("*"):
                if path.is_file() and path.suffix in EXTENSIONS:
                    yield path


def main(argv: list[str]) -> int:
    here = Path(__file__).resolve().parent
    args = argv[1:]

    root_args = args[1:] if len(args) > 1 else DEFAULT_ROOTS
    roots = [Path(r) if Path(r).is_absolute() else here / r for r in root_args]


    macros = DEFAULT_MACROS

    if not isinstance(macros, dict):
        print("error: macro file must contain a JSON object", file=sys.stderr)
        return 1

    total_changes = 0
    files_changed = 0
    for path in iter_files(roots):
        original = path.read_text()
        rel = path.relative_to(here) if path.is_relative_to(here) else path
        patched, changes = patch_text(original, macros, str(rel))
        if changes:
            path.write_text(patched)
            files_changed += 1
            total_changes += len(changes)
            print(f"{rel}")
            for name, old, new in changes:
                print(f"  {name}: {old!r} -> {new!r}")

    print(f"\n{total_changes} replacement(s) across {files_changed} file(s).")
    return 0


if __name__ == "__main__":
    sys.exit(main(sys.argv))
```

3 Pin Input / Output Table

The table below lists every pin used across the Arduino Uno and the ESP8266 NodeMCU, the component (software module) that owns the pin, and the role that pin plays in the system.

Board	Direction	Pin	Component	Function
Arduino	OUTPUT	D10	Light (Red LED)	PWM drive for the red channel of the RGB lamp (`analogWrite`).
Arduino	OUTPUT	D11	Light (Green LED)	PWM drive for the green channel of the RGB lamp.

Board	Direction	Pin	Component	Function
Arduino	OUTPUT	D3	Light (Blue LED)	PWM drive for the blue channel of the RGB lamp.
Arduino	INPUT	A0	Sensor (Analog)	Reads the analog motion / proximity sensor; lamp is considered active when the value crosses `SENSOR_THRESHOLD` (512).
Arduino	INPUT	D13	Sensor (Digital)	Alternative digital sensor input (used when `SENSOR_MODE = "DIGITAL"`).
Arduino	OUTPUT	D12	Internet (TX-ADD)	Pulses HIGH for ∼1 s to tell the local ESP8266 to broadcast an "ADD" request, marking adjacent lamps as active on the shared network list.
Arduino	OUTPUT	D5	Internet (TX-REMOVE)	Pulses HIGH for ∼1 s to tell the local ESP8266 to send a "REMOVE" acknowledgement, removing this lamp from the network active list.
Arduino	INPUT	D6	Internet (RX)	Reads the activation line from the local ESP8266; HIGH means this lamp's index appears in the network active list and the lamp should turn on.
ESP8266	INPUT	GPIO4 (D2)	Internet (RX-ADD)	Receives the Arduino's "add adjacent lamps" pulse; on a rising edge the ESP sends `ADD <n>` for each adjacent index over Wi-Fi.
ESP8266	INPUT	GPIO16 (D0)	Internet (RX-REMOVE)	Receives the Arduino's "remove me" pulse; the ESP sends `REMOVE <LIGHT_INDEX>` over Wi-Fi to clear itself from the active list.
ESP8266	OUTPUT	GPIO14 (D5)	Internet (TX)	Drives the Arduino's RX line HIGH whenever this lamp's `LIGHT_INDEX` is present in the broadcast active list (i.e. this lamp should turn on).

Table 1: Pin assignments across the Arduino Uno and ESP8266 NodeMCU for one lamp node.

4 Git Commits Tied to Each Feature

The table below maps each major feature of the project to the git commits that introduced or finalized it. Commit hashes are short SHAs from the project repository.

Feature	Commit(s)	Description
Project skeleton / PlatformIO scaffold	`8fd7f04` (Initial commit)	Created the PlatformIO project layout, `.gitignore`, and stub `src/main.cpp`.
Module skeletons (Internet, Light, Sensor, SignalCoordinator)	`c5e277e` (skeleton)	Stubbed out the `Internet`, `Light`, `Sensor` and `SignalCoordinator` libraries with empty class methods.

Feature	Commit(s)	Description
Skeleton wiring & `Utils` helper	`92fed2d` (Skeleton done)	Connected the modules into `main.cpp` via `begin()`/`update()`, added the `Utils` library, and fleshed out an early `SignalCoordinator`.
RGB pin macros	`8542f63` (Put RGB macros in Light.h)	Centralized the `RED_PIN`, `GREEN_PIN`, `BLUE_PIN` pin numbers in `Light.h`.
RGB lamp output (Light module)	`144188f` (Completed Light.cpp)	Implemented `Light::begin` (`pinMode` OUTPUT) and `Light::setColor` using `analogWrite` on the three PWM pins.
Repository restructure into Arduino + ESP subprojects	`2c1c8f3` (Added ESP subrepo)	Split the codebase into `ardu code/` and `ESP_code_EE120_final/` so the Uno and ESP8266 firmware can be built as separate PlatformIO projects.
Cross-platform macro patching tool	`109c159` (Added gabage slop macro patching)	Added `macrocompile.py`, which rewrites `#define` values across both subprojects from a single source of truth so pin numbers stay in sync between the Arduino and ESP code.
ESP8266 Wi-Fi networking (router/client, ADD/REMOVE/LIST protocol)	`38517a6` (ESP CODE)	Implemented the ESP firmware: SoftAP router mode with `WiFiServer`/client list, station-mode clients, and the `ADD <n>` / `REMOVE <n>` / `LIST ...` text protocol. Also added the matching Arduino-side `txMessage` / `rxMessage` / `txDeleteMessage` signal lines.
Analog sensor mode	`eb52ebb`, `2b9086c` (Update Sensor.cpp)	Added analog sensor support with a configurable threshold and active-high / active-low logic; later tweaked the comparison direction.
Per-lamp adjacency macros	`24aa73e` (Pity macros for Nick S.)	Added the `LAMP_INDEX`-driven adjacency / router selection to `macrocompile.py` (each lamp's neighbours are auto-set), and matching ESP-side `ADJACENT`/`LIGHT_INDEX` macros.
End-to-end integration polish	`dd32614` (Finishing touches)	Final wiring of the Arduino main loop: TX-add / TX-remove pulse timing, the 10-second active timer, and the ack-back "remove me" message after receiving an active signal.
Final flashed configuration	`6446a32` (Flashed final 3)	Last on-device tuning pass before demo — pin number cleanup, sensor calibration, and merged `Internet` / `SignalCoordinator` edits used in the final flashed image.

Table 2: Mapping of features to the commits that introduced or finalized them.